

Справочник по C++ Thread Library

Г.1. Заголовок <chrono>

Заголовок <chrono> предоставляет классы для представления моментов времени, продолжительности `duration` и часов, которые служат источником объектов `time_point`. В каждом классе часов есть статический компонент `is_steady`, показывающий, являются ли данные часы *стабильными* (это значит, что их не нужно подводить). Единственные гарантированно стабильные часы представлены классом `std::chrono::steady_clock`.

Содержимое заголовка

```
namespace std
{
    namespace chrono
    {
        template<typename Rep,typename Period = ratio<1>>
        class duration;
        template<
            typename Clock,
            typename Duration = typename Clock::duration>
        class time_point;
        class system_clock;
        class steady_clock;
        typedef unspecified-clock-type high_resolution_clock;
    }
}
```

Г.1.1. Шаблон класса `std::chrono::duration`

Шаблон класса `std::chrono::duration` предназначен для представления продолжительности. Параметры шаблона `Rep` и `Period` являются соответственно типами данных для хранения значения продолжительности и экземпляра шаблона класса `std::ratio`, которым задается временной промежуток (в долях секунды) между последовательными тактами. Например, `std::chrono::duration<int, std::milli>` определяет количество миллисекунд, которое может храниться в значении типа `int`, `std::chrono::duration<short, std::ratio<1,50>>` — количество 50-х долей секунды, которое может храниться в значении типа `short`, а `std::chrono::duration<long long, std::ratio<60,1>>` — количество минут, которое может храниться в значении типа `long long`.

Определение класса

```
template <class Rep, class Period=ratio<1> >
class duration
{
public:
    typedef Rep rep;
    typedef Period period;

    constexpr duration() = default;
    ~duration() = default;

    duration(const duration&) = default;
    duration& operator=(const duration&) = default;

    template <class Rep2>
    constexpr explicit duration(const Rep2& r);

    template <class Rep2, class Period2>
    constexpr duration(const duration<Rep2, Period2>& d);

    constexpr rep count() const;
    constexpr duration operator+() const;
    constexpr duration operator-() const;
    duration& operator++();
    duration operator++(int);
    duration& operator--();
    duration operator--(int);
    duration& operator+=(const duration& d);
    duration& operator-=(const duration& d);
    duration& operator*=(const rep& rhs);
    duration& operator/=(const rep& rhs);
    duration& operator%=(const rep& rhs);
    duration& operator%=(const duration& rhs);
    static constexpr duration zero();
    static constexpr duration min();
    static constexpr duration max();
};
```

```

template <class Rep1, class Period1, class Rep2, class Period2>
constexpr bool operator==(
    const duration<Rep1, Period1>& lhs,
    const duration<Rep2, Period2>& rhs);

template <class Rep1, class Period1, class Rep2, class Period2>
constexpr bool operator!=(
    const duration<Rep1, Period1>& lhs,
    const duration<Rep2, Period2>& rhs);

template <class Rep1, class Period1, class Rep2, class Period2>
constexpr bool operator<(
    const duration<Rep1, Period1>& lhs,
    const duration<Rep2, Period2>& rhs);

template <class Rep1, class Period1, class Rep2, class Period2>
constexpr bool operator<=(
    const duration<Rep1, Period1>& lhs,
    const duration<Rep2, Period2>& rhs);

template <class Rep1, class Period1, class Rep2, class Period2>
constexpr bool operator>(
    const duration<Rep1, Period1>& lhs,
    const duration<Rep2, Period2>& rhs);

template <class Rep1, class Period1, class Rep2, class Period2>
constexpr bool operator>=(
    const duration<Rep1, Period1>& lhs,
    const duration<Rep2, Period2>& rhs);

template <class ToDuration, class Rep, class Period>
constexpr ToDuration duration_cast(const duration<Rep, Period>& d);

```

Требования

Rep должен быть встроенным числовым или подобным числовому типом, определенным пользователем. Period должен быть экземпляром, созданным по шаблону `std::ratio<>`.

Псевдоним типа `std::chrono::duration::rep`

Это псевдоним типа для хранения числа тактов в значении `duration`.

Объявление

```
typedef Rep rep;
```

rep — тип значения, используемого для хранения внутреннего представления объекта `duration`.

Псевдоним типа `std::chrono::duration::period`

Это псевдоним типа для создания экземпляра по шаблону класса `std::ratio`, с помощью которого задается количество долей секунды, представляемых счетчиком про-

должительности. Например, если `period` — это `std::ratio<1,50>`, то объект `duration`, для которого `count()` возвращает значение `N`, представляет `N` 50-х долей секунды.

Объявление

```
typedef Period period;
```

Конструктор по умолчанию `std::chrono::duration`

Создает экземпляр `std::chrono::duration` со значением по умолчанию.

Объявление

```
constexpr duration() = default;
```

Результат

Внутреннее значение `duration` (типа `rep`) инициализируется значением по умолчанию.

Конструктор преобразования из значения счетчика `std::chrono::duration`

Создает экземпляр `std::chrono::duration` с заданным значением счетчика.

Объявление

```
template <class Rep2>
constexpr explicit duration(const Rep2& r);
```

Результат

Внутреннее значение объекта `duration` инициализируется значением `static_cast<rep>(r)`.

Требования

Этот конструктор участвует в разрешении переопределения при условии, что `Rep2` может быть подразумеваемо преобразован в `Rep`, а также что либо `Rep` является типом с плавающей точкой, либо `Rep2` не является типом с плавающей точкой.

Постусловие

```
this->count()==static_cast<rep>(r)
```

Конструктор преобразования `std::chrono::duration` из другого значения `std::chrono::duration`

Создает экземпляр `std::chrono::duration` путем масштабирования значения счетчика другого объекта `std::chrono::duration`.

Объявление

```
template <class Rep2, class Period2>
constexpr duration(const duration<Rep2,Period2>& d);
```

Результат

Внутреннее значение объекта `duration` инициализируется значением `duration_cast<duration<Rep,Period>>(d).count()`.

Требования

Этот конструктор участвует в разрешении переопределения при условии, что `Rep` является типом с плавающей точкой или `Rep2` не является типом с плавающей точкой, а `Period2` является целым числом, кратным `Period` (то есть `ratio_divide<Period2,Period>::den==1`). Это позволяет избежать случайного усечения (и соответствующей потери точности) из-за сохранения продолжительности с меньшими периодами в переменной, представляющей продолжительность с большим периодом.

Постусловие

```
this->count()==duration_cast<duration<Rep,Period>>(d).count()
```

Примеры

```
duration<int,ratio<1,1000>> ms(5);  ← Пять миллисекунд
duration<int,ratio<1,1>> s(ms);      ← Ошибка: сохранить ms
                                     в виде целых секунд невозможно
duration<double,ratio<1,1>> s2(ms); ← Все в порядке: s2.count()==0.005
duration<int,ratio<1,1000000>> us(ms); ← Все в порядке: us.count()==5000
```

Компонентная функция `std::chrono::duration::count`

Извлекает значение продолжительности.

Объявление

```
constexpr rep count() const;
```

Возвращает

Внутреннее значение объекта `duration` в виде значения типа `rep`.

Унарный оператор
плюс `std::chrono::duration::operator+`

Пустая операция: просто возвращает копию `*this`.

Объявление

```
constexpr duration operator+() const;
```

Возвращает

```
*this
```

Унарный оператор
минус `std::chrono::duration::operator-`

Возвращает продолжительность таким образом, что значением `count()` становится отрицательное значение `this->count()`.

Объявление

```
constexpr duration operator-() const;
```

Возвращает

```
duration(-this->count());
```

Оператор прединкремента `std::chrono::duration::operator++`

Увеличивает значение внутреннего счетчика на единицу.

Объявление

```
duration& operator++();
```

Результат

```
++this->internal_count;
```

Возвращает

```
*this
```

Оператор постинкремента `std::chrono::duration::operator++`

Увеличивает значение внутреннего счетчика на единицу и возвращает значение `*this`, предшествовавшее увеличению.

Объявление

```
duration operator++(int);
```

Результат

```
duration temp(*this);
```

```
++(*this);
```

```
return temp;
```

Оператор преддекремента `std::chrono::duration::operator--`

Уменьшает значение внутреннего счетчика на единицу.

Объявление

```
duration& operator--();
```

Результат

```
--this->internal_count;
```

Возвращает

```
*this
```

Оператор постдекремента `std::chrono::duration::operator--`

Уменьшает значение внутреннего счетчика на единицу и возвращает значение `*this`, предшествовавшее уменьшению.

Объявление

```
duration operator--(int);
```

Результат

```
duration temp(*this);
```

```
--(*this);
```

```
return temp;
```

Составной оператор присваивания `std::chrono::duration::operator+=`

Прибавляет значение счетчика другого объекта `duration` к значению внутреннего счетчика для `*this`.

Объявление

```
duration& operator+=(duration const& other);
```

Результат

```
internal_count+=other.count();
```

Возвращает

`*this`

Составной оператор присваивания `std::chrono::duration::operator-=`

Вычитает значение счетчика другого объекта `duration` из значения внутреннего счетчика для `*this`.

Объявление

```
duration& operator-=(duration const& other);
```

Результат

```
internal_count-=other.count();
```

Возвращает

`*this`

Составной оператор присваивания `std::chrono::duration::operator*=`

Умножает значение внутреннего счетчика для `*this` на заданное значение.

Объявление

```
duration& operator*=(rep const& rhs);
```

Результат

```
internal_count*=rhs;
```

Возвращает

`*this`

Составной оператор присваивания `std::chrono::duration::operator/=`

Делит значение внутреннего счетчика `*this` на указанное значение.

Объявление

```
duration& operator/=(rep const& rhs);
```

Результат

```
internal_count/=rhs;
```

Возвращает

`*this`

Составной оператор присваивания `std::chrono::duration::operator%=`

Подгоняет значение внутреннего счетчика для `*this` под остаток от деления на заданное значение.

Объявление

```
duration& operator%=(rep const& rhs);
```

Результат

```
internal_count%=rhs;
```

Возвращает

```
*this
```

Составной оператор присваивания `std::chrono::duration::operator%=`

Подгоняет значение внутреннего счетчика для `*this` под остаток от деления на заданное значение на значение счетчика другого объекта `duration`.

Объявление

```
duration& operator%=(duration const& rhs);
```

Результат

```
internal_count%=rhs.count();
```

Возвращает

```
*this
```

Статическая компонентная функция `std::chrono::duration::zero`

Возвращает объект `duration`, представляющий нулевое значение.

Объявление

```
constexpr duration zero();
```

Возвращает

```
duration(duration_values<rep>::zero());
```

Статическая компонентная функция `std::chrono::duration::min`

Возвращает объект `duration`, содержащий минимальное возможное значение для указанного экземпляра.

Объявление

```
constexpr duration min();
```

Возвращает

```
duration(duration_values<rep>::min());
```

Статическая компонентная функция `std::chrono::duration::max`

Возвращает объект `duration`, содержащий максимальное возможное значение для указанного экземпляра.

Объявление

```
constexpr duration max();
```

Возвращает

```
duration(duration_values<rep>::max());
```

Оператор сравнения для определения равенства `std::chrono::duration`

Сравнивает два объекта `duration` для определения равенства, даже если у них разные представления `n` (или) периоды.

Объявление

```
template <class Rep1, class Period1, class Rep2, class Period2>  
constexpr bool operator==(  
    const duration<Rep1, Period1>& lhs,  
    const duration<Rep2, Period2>& rhs);
```

Требования

Подразумеваемое преобразование должно быть либо `lhs` в `rhs`, либо наоборот. Если же такое преобразование невозможно ни для одной из частей или же они являются несхожими экземплярами `duration`, но каждая из них подразумеваемо может быть преобразована в другую, значит, выражение сформировано неверно.

Результат

Если `CommonDuration` является синонимом для `std::common_type<duration<Rep1, Period1>, duration<Rep2, Period2>>::type`, то `lhs==rhs` возвращает `CommonDuration(lhs).count()==CommonDuration(rhs).count()`.

Оператор сравнения для определения неравенства `std::chrono::duration`

Сравнивает два объекта `duration` для определения неравенства, даже если у них разные представления `n` (или) периоды.

Объявление

```
template <class Rep1, class Period1, class Rep2, class Period2>  
constexpr bool operator!=(  
    const duration<Rep1, Period1>& lhs,  
    const duration<Rep2, Period2>& rhs);
```

Требования

Должно быть подразумеваемое преобразование либо `lhs` в `rhs`, либо наоборот. Если же такое преобразование невозможно ни для одной из частей или же они

являются несхожими экземплярами `duration`, но каждая из них подразумеваемо может быть преобразована в другую, значит, выражение сформировано неверно.

Возвращает

`!(lhs==rhs)`

Оператор сравнения «меньше» `std::chrono::duration`

Сравнивает два объекта `duration` для определения, не является ли один меньше другого, даже если у них разные представления и (или) периоды.

Объявление

```
template <class Rep1, class Period1, class Rep2, class Period2>
constexpr bool operator<(
    const duration<Rep1, Period1>& lhs,
    const duration<Rep2, Period2>& rhs);
```

Требования

Должно быть подразумеваемое преобразование либо `lhs` в `rhs`, либо наоборот. Если же такое преобразование невозможно ни для одной из частей или же они являются несхожими экземплярами `duration`, но каждая из них подразумеваемо может быть преобразована в другую, значит, выражение сформировано неверно.

Результат

Если `CommonDuration` является синонимом для `std::common_type<duration<Rep1, Period1>, duration<Rep2, Period2>>::type`, то `lhs<rhs` возвращает `CommonDuration(lhs).count()<CommonDuration(rhs).count()`.

Оператор сравнения «больше» `std::chrono::duration`

Сравнивает два объекта `duration` для определения, не является ли один больше другого, даже если у них разные представления и (или) периоды.

Объявление

```
template <class Rep1, class Period1, class Rep2, class Period2>
constexpr bool operator>(
    const duration<Rep1, Period1>& lhs,
    const duration<Rep2, Period2>& rhs);
```

Требования

Должно быть подразумеваемое преобразование либо `lhs` в `rhs`, либо наоборот. Если же такое преобразование невозможно ни для одной из частей или же они являются несхожими экземплярами `duration`, но каждая из них подразумеваемо может быть преобразована в другую, значит, выражение сформировано неверно.

Возвращает

`rhs<lhs`

Оператор сравнения «меньше или равно» `std::chrono::duration`

Сравнивает два объекта `duration` для определения, не является ли один меньше другого или равным ему, даже если у них разные представления `n` (или) периоды.

Объявление

```
template <class Rep1, class Period1, class Rep2, class Period2>
constexpr bool operator<=(
    const duration<Rep1, Period1>& lhs,
    const duration<Rep2, Period2>& rhs);
```

Требования

Должно быть подразумеваемое преобразование либо `lhs` в `rhs`, либо наоборот. Если же такое преобразование невозможно ни для одной из частей или же они являются несхожими экземплярами `duration`, но каждая из них подразумеваемо может быть преобразована в другую, значит, выражение сформировано неверно.

Возвращает

`!(rhs<lhs)`

Оператор сравнения «больше или равно» `std::chrono::duration`

Сравнивает два объекта `duration` для определения, не является ли один больше другого или равным ему, даже если у них разные представления `n` (или) периоды.

Объявление

```
template <class Rep1, class Period1, class Rep2, class Period2>
constexpr bool operator>=(
    const duration<Rep1, Period1>& lhs,
    const duration<Rep2, Period2>& rhs);
```

Требования

Должно быть подразумеваемое преобразование либо `lhs` в `rhs`, либо наоборот. Если же такое преобразование невозможно ни для одной из частей или же они являются несхожими экземплярами `duration`, но каждая из них подразумеваемо может быть преобразована в другую, значит, выражение сформировано неверно.

Возвращает

`!(lhs<rhs)`

Некомпонентная функция `std::chrono::duration_cast`

Выполняет явное преобразование объекта `std::chrono::duration` в заданный экземпляр `std::chrono::duration`.

Объявление

```
template <class ToDuration, class Rep, class Period>
constexpr ToDuration duration_cast(const duration<Rep, Period>& d);
```

Требования

Объект `ToDuration` должен быть экземпляром класса `std::chrono::duration`.

Возвращает

Продолжительность `d`, преобразованная в тип продолжительности, заданный объектом `ToDuration`. Преобразование выполняется таким образом, чтобы минимизировать любые вызванные им потери точности между различными масштабами и типами представления.

Г.1.2. Шаблон класса `std::chrono::time_point`

Шаблон класса `std::chrono::time_point` позволяет представить момент времени, измеренный по конкретным часам. Задается в виде продолжительности со времени начала эпохи отсчета этих конкретных часов. Параметр шаблона `Clock` идентифицирует часы (у разных часов должен быть уникальный тип), а параметр шаблона `Duration` является типом, используемым для измерения продолжительности со времени начала эпохи и должен быть экземпляром шаблона класса `std::chrono::duration`. По умолчанию значением `Duration` должен быть исходный тип продолжительности шаблона `Clock`.

Определение класса

```
template <class Clock, class Duration = typename Clock::duration>
class time_point
{
public:
    typedef Clock clock;
    typedef Duration duration;
    typedef typename duration::rep rep;
    typedef typename duration::period period;

    time_point();
    explicit time_point(const duration& d);

    template <class Duration2>
    time_point(const time_point<clock, Duration2>& t);

    duration time_since_epoch() const;

    time_point& operator+=(const duration& d);
    time_point& operator-=(const duration& d);

    static constexpr time_point min();
    static constexpr time_point max();
};
```

Конструктор по умолчанию `std::chrono::time_point`

Создает объект `time_point`, представляющий продолжительность с начала эпохи отсчета, задаваемой связанным шаблоном `Clock`; внутренняя продолжительность инициализируется значением `Duration::zero()`.

Объявление

```
time_point();
```

Постусловие

Для только что созданного переходного объекта `time_point` по имени `tp` `tp.time_since_epoch()==tp::duration::zero()`.

Конструктор продолжительности `std::chrono::time_point`

Создает объект `time_point`, представляющий продолжительность с начала эпохи отсчета, задаваемой связанным шаблоном `clock`.

Объявление

```
explicit time_point(const duration& d);
```

Постусловие

Для объекта `time_point` по имени `tp`, созданного с использованием конструктора `tp(d)` для некоторой продолжительности `d`, `tp.time_since_epoch()==d`.

Конструктор преобразования `std::chrono::time_point`

Создает объект `time_point` из другого объекта `time_point` с таким же типом `clock`, но с другим типом `Duration`.

Объявление

```
template <class Duration2>  
time_point(const time_point<clock, Duration2>& t);
```

Требования

Тип `Duration2` подразумевается должен преобразовываться в `Duration`.

Результат

Как при вычислении `time_point(t.time_since_epoch())`.

Значение, возвращаемое функцией `t.time_since_epoch()`, проходит подразумеваемое преобразование в объект типа `Duration`, и это значение сохраняется в только что созданном объекте `time_point`.

Компонентная

функция `std::chrono::time_point::time_since_epoch`

Извлекает продолжительность с начала эпохи отсчета для конкретного объекта `time_point`.

Объявление

```
duration time_since_epoch() const;
```

Возвращает

Значение `duration`, хранящееся в `*this`.

Составной оператор присваивания `std::chrono::time_point::operator+=`

Прибавляет заданную продолжительность к значению, сохраненному в указанном объекте `time_point`.

Объявление

```
time_point& operator+=(const duration& d);
```

Результат

Прибавляет `d` к внутреннему объекту продолжительности `*this`, как при вычислении `this->internal_duration += d`.

Возвращает

`*this`

Составной оператор присваивания `std::chrono::time_point::operator-=`

Вычитает заданную продолжительность из значения, сохраненного в указанном объекте `time_point`.

Объявление

```
time_point& operator-=(const duration& d);
```

Результат

Вычитает `d` из внутреннего объекта продолжительности `*this`, как при вычислении `this->internal_duration -= d`.

Возвращает

`*this`

Статическая компонентная функция `std::chrono::time_point::min`

Получает объект `time_point`, представляющий минимально возможное для данного типа значение.

Объявление

```
static constexpr time_point min();
```

Возвращает

```
time_point(time_point::duration::min())
```

Статическая компонентная функция `std::chrono::time_point::max`

Получает объект `time_point`, представляющий максимально возможное для данного типа значение.

Объявление

```
static constexpr time_point max();
```

Возвращает

```
time_point(time_point::duration::max())
```

Г.1.3. Класс `std::chrono::system_clock`

Класс `std::chrono::system_clock` предоставляет средства для получения текущего физического времени. Текущее время можно получить, вызвав функцию `std::chrono::system_clock::now()`. Экземпляры класса `std::chrono::system_clock::time_point` с помощью функций `std::chrono::system_clock::to_time_t()` и `std::chrono::system_clock::to_time_point()` можно преобразовать в `time_t` и обратно. Системные часы часто нестабильны, поэтому при последующем обращении к `std::chrono::system_clock::now()` можно вернуть время, предшествующее времени, возвращенному предыдущим вызовом (например, если часы операционной системы были вручную переведены или синхронизированы с внешними часами).

Определение класса

```
class system_clock
{
public:
    typedef unspecified-integral-type rep;
    typedef std::ratio<unspecified,unspecified> period;
    typedef std::chrono::duration<rep,period> duration;
    typedef std::chrono::time_point<system_clock> time_point;
    static const bool is_steady=unspecified;

    static time_point now() noexcept;
    static time_t to_time_t(const time_point& t) noexcept;
    static time_point from_time_t(time_t t) noexcept;
};
```

Псевдоним типа `std::chrono::system_clock::rep`

Псевдоним целочисленного типа, используемого для хранения количества тактов в значении продолжительности `duration`.

Объявление

```
typedef unspecified-integral-type rep;
```

Псевдоним типа `std::chrono::system_clock::period`

Псевдоним типа для экземпляра шаблона класса `std::ratio`, задающего наименьшее количество секунд (или долей секунды) между различными значениями `duration` или `time_point`. Псевдоним `period` определяет точность часов, а не частоту тактов.

Объявление

```
typedef std::ratio<unspecified,unspecified> period;
```

Псевдоним типа `std::chrono::system_clock::duration`

Экземпляр шаблона класса `std::chrono::duration`, способный сохранять разницу между любыми двумя моментами времени, возвращаемыми общесистемными часами реального времени.

Объявление

```
typedef std::chrono::duration<
std::chrono::system_clock::rep,
std::chrono::system_clock::period> duration;
```

Псевдоним типа `std::chrono::system_clock::time_point`

Экземпляр шаблона класса `std::chrono::time_point`, способный сохранять моменты времени, возвращаемые общесистемными часами реального времени.

Объявление

```
typedef std::chrono::time_point<std::chrono::system_clock> time_point;
```

Статическая компонентная функция `std::chrono::system_clock::now`

Получает текущее время от общесистемных часов реального времени.

Объявление

```
time_point now() noexcept;
```

Возвращает

Объект `time_point`, представляющий текущее время общесистемных часов реального времени.

Выдает

Исключение `std::system_error` в случае ошибки.

Статическая компонентная функция `std::chrono::system_clock::to_time_t`

Выполняет преобразование экземпляра `time_point` в `time_t`.

Объявление

```
time_t to_time_t(time_point const& t) noexcept;
```

Возвращает

Значение `time_t`, представляющее тот же момент времени, что и `t`, округленный или усеченный с точностью до секунд.

Выдает

Исключение `std::system_error` в случае ошибки.

Статическая компонентная функция `std::chrono::system_clock::from_time_t`

Выполняет преобразование экземпляра `time_t` в `time_point`.

Объявление

```
time_point from_time_t(time_t const& t) noexcept;
```

Возвращает

Значение `time_point`, представляющее тот же момент времени, что и `t`.

Выдает

Исключение `std::system_error` в случае ошибки.

Г.1.4. Класс `std::chrono::steady_clock`

Класс `std::chrono::steady_clock` предоставляет доступ к общесистемным стабильным часам. Текущее время можно получить, вызвав функцию `std::chrono::steady_clock::now()`. Фиксированной связи между значениями, возвращенными `std::chrono::steady_clock::now()`, и физическими часами не существует. Стабильные часы нельзя перевести назад, поэтому, если один вызов `std::chrono::steady_clock::now()` произошел до другого вызова `std::chrono::steady_clock::now()`, второй вызов должен вернуть значение времени, равное тому, что было возвращено первым вызовом, или более позднее. Часы идут как можно равномернее.

Определение класса

```
class steady_clock
{
public:
    typedef unspecified-integral-type rep;
    typedef std::ratio<
        unspecified,unspecified> period;
    typedef std::chrono::duration<rep,period> duration;
    typedef std::chrono::time_point<steady_clock>
        time_point;
    static const bool is_steady=true;

    static time_point now() noexcept;
};
```

Псевдоним типа `std::chrono::steady_clock::rep`

Этот псевдоним типа предназначен для целочисленного типа, используемого для хранения числа тактов в значении `duration`.

Объявление

```
typedef unspecified-integral-type rep;
```


Псевдоним типа `std::chrono::steady_clock::period`

Этот псевдоним типа предназначен для экземпляров шаблона класса `std::ratio`, определяющих наименьшее количество секунд (или долей секунды) между различными значениями `duration` или `time_point`. Псевдоним `period` определяет точность часов, а не частоту тактов.

Объявление

```
typedef std::ratio<unspecified,unspecified> period;
```

Псевдоним типа `std::chrono::steady_clock::duration`

Экземпляр шаблона класса `std::chrono::duration`, способный сохранять разницу между любыми двумя моментами времени, возвращаемыми общесистемными стабильными часами.

Объявление

```
typedef std::chrono::duration<
std::chrono::steady_clock::rep,
std::chrono::steady_clock::period> duration;
```

Псевдоним типа `std::chrono::steady_clock::time_point`

Экземпляр шаблона класса `std::chrono::time_point`, способный сохранять моменты времени, возвращаемые общесистемными стабильными часами.

Объявление

```
typedef std::chrono::time_point<std::chrono::steady_clock> time_point;
```

Статическая компонентная функция `std::chrono::steady_clock::now`

Получает текущее время от общесистемных стабильных часов.

Объявление

```
time_point now() noexcept;
```

Возвращает

Объект `time_point`, представляющий текущее время общесистемных стабильных часов.

Выдает

Исключение `std::system_error` в случае ошибки.

Синхронизация

Если один вызов `std::chrono::steady_clock::now()` происходит раньше другого, то объект `time_point`, возвращенный первым вызовом, должен быть меньше или равен моменту времени, возвращенному вторым вызовом.

Г.1.5. Псевдоним типа `std::chrono::high_resolution_clock`

Класс `std::chrono::high_resolution_clock` предоставляет доступ к общесистемным часам с наивысшим разрешением. Как и для всех остальных часов, текущее время можно получить, вызвав функцию `std::chrono::high_resolution_clock::now()`. Идентификатор `std::chrono::high_resolution_clock` может быть псевдонимом типа для класса `std::chrono::system_clock` или класса `std::chrono::steady_clock`, или может быть отдельным типом.

Хотя у типа `std::chrono::high_resolution_clock` самое высокое разрешение среди всех предоставляемых библиотекой часов, вызов функции `std::chrono::high_resolution_clock::now()` все же занимает определенное время. При хронометрировании скоротечных операций нужно брать в расчет издержки на вызов `std::chrono::high_resolution_clock::now()`.

Определение класса

```
class high_resolution_clock
{
public:
    typedef unspecified-integral-type rep;
    typedef std::ratio<
        unspecified,unspecified> period;
    typedef std::chrono::duration<rep,period> duration;
    typedef std::chrono::time_point<
        unspecified> time_point;
    static const bool is_steady=unspecified;

    static time_point now() noexcept;
};
```

Г.2. Заголовок <condition_variable>

Заголовок <condition_variable> предоставляет условные переменные, являющиеся механизмами синхронизации базового уровня, позволяющими потоку быть заблокированным, пока не поступит уведомление о выполнении определенного условия или не истечет время ожидания, заданное тайм-аутом.

Содержимое заголовка

```
namespace std
{
    enum class cv_status { timeout, no_timeout };

    class condition_variable;
    class condition_variable_any;
}
```

Г.2.1. Класс `std::condition_variable`

Класс `std::condition_variable` позволяет потоку ждать, пока условие не получит значение `true`. Экземпляры `std::condition_variable` не являются `CopyAssignable`, `CopyConstructible`, `MoveAssignable` или `MoveConstructible`.

Определение класса

```

class condition_variable
{
public:
    condition_variable();
    ~condition_variable();

    condition_variable(condition_variable const& ) = delete;
    condition_variable& operator=(condition_variable const& ) = delete;

    void notify_one() noexcept;
    void notify_all() noexcept;

    void wait(std::unique_lock<std::mutex>& lock);

    template <typename Predicate>
    void wait(std::unique_lock<std::mutex>& lock, Predicate pred);

    template <typename Clock, typename Duration>
    cv_status wait_until(
        std::unique_lock<std::mutex>& lock,
        const std::chrono::time_point<Clock, Duration>& absolute_time);

    template <typename Clock, typename Duration, typename Predicate>
    bool wait_until(
        std::unique_lock<std::mutex>& lock,
        const std::chrono::time_point<Clock, Duration>& absolute_time,
        Predicate pred);

    template <typename Rep, typename Period>
    cv_status wait_for(
        std::unique_lock<std::mutex>& lock,
        const std::chrono::duration<Rep, Period>& relative_time);

    template <typename Rep, typename Period, typename Predicate>
    bool wait_for(
        std::unique_lock<std::mutex>& lock,
        const std::chrono::duration<Rep, Period>& relative_time,
        Predicate pred);
};

void notify_all_at_thread_exit(condition_variable&, unique_lock<mutex>);

```

Конструктор по умолчанию std::condition_variable

Создает объект `std::condition_variable`.

Объявление

```
condition_variable();
```

Результат

Создает новый экземпляр `std::condition_variable`.

Выдает

Исключение `std::system_error`, если условная переменная не была создана.

Деструктор std::condition_variable

Уничтожает объект `std::condition_variable`.

Объявление

```
~condition_variable();
```

Предусловия

Отсутствие потоков, заблокированных на `*this` в вызове `wait()`, `wait_for()` или `wait_until()`.

Результат

Уничтожает `*this`.

Выдает

Ничего не выдает.

Компонентная функция std::condition_variable::notify_one

Пробуждает один из потоков, ожидающих условной переменной `std::condition_variable`.

Объявление

```
void notify_one() noexcept;
```

Результат

Пробуждает в точке вызова один из потоков, ожидающих `*this`. При отсутствии ожидающих потоков вызов не даст никакого результата.

Выдает

Исключение `std::system_error`, если результат не может быть достигнут.

Синхронизация

Вызовы `notify_one()`, `notify_all()`, `wait()`, `wait_for()` и `wait_until()` в отношении одного и того же экземпляра `std::condition_variable` выстраиваются в последовательность. Вызов `notify_one()` или `notify_all()` разбудит только те потоки, которые приступили к ожиданию до этого вызова.

Компонентная функция std::condition_variable::notify_all

Пробуждает все потоки, ожидающие на `std::condition_variable`.

Объявление

```
void notify_all() noexcept;
```

Результат

Пробуждает в точке вызова все потоки, ожидающие на `*this`. При отсутствии ожидающих потоков вызов не даст никакого результата.

Выдает

Исключение `std::system_error`, если результат не может быть достигнут.

Синхронизация

Вызовы `notify_one()`, `notify_all()`, `wait()`, `wait_for()` и `wait_until()` в отношении одного и того же экземпляра `std::condition_variable` выстраиваются в последовательность. Вызов `notify_one()` или `notify_all()` разбудит только те потоки, которые приступили к ожиданию до этого вызова.

Компонентная функция `std::condition_variable::wait`

Ожидает пробуждения `std::condition_variable` вызовом `notify_one()/notify_all()` или ложного пробуждения.

Объявление

```
void wait(std::unique_lock<std::mutex>& lock);
```

Предусловия

`lock.owns_lock()` возвращает `true`, и блокировка принадлежит вызывающему потоку.

Результат

Атомарно снимает блокировку с предоставленного объекта `lock` и блокирует поток, пока он не будет разбужен вызовом `notify_one()` или `notify_all()` из другого потока или же пока не произойдет ложное пробуждение потока. Перед возвратом управления из вызова `wait()` объект `lock` снова блокируется.

Выдает

Исключение `std::system_error`, если результат не может быть достигнут. Если объект `lock` был разблокирован в ходе вызова `wait()`, то на выходе он снова блокируется, даже если выход из функции происходит по выдаче исключения.

ПРИМЕЧАНИЕ

Ложные пробуждения означают, что вызывавший функцию `wait()` поток можно разбудить, даже если ни один из потоков не вызывал функцию `notify_one()` или `notify_all()`. Поэтому по возможности рекомендуется применять переопределение функции `wait()`, принимающее предикат. Или же рекомендуется вызывать `wait()` в цикле, проверяющем предикат, связанный с условной переменной.

Синхронизация

Вызовы `notify_one()`, `notify_all()`, `wait()`, `wait_for()` и `wait_until()` в отношении одного и того же экземпляра `std::condition_variable` выстраиваются в последовательность. Вызов `notify_one()` или `notify_all()` разбудит только те потоки, которые приступили к ожиданию до этого вызова.

Переопределение компонентной функции `std::condition_variable::wait`, принимающее предикат

Ожидает пробуждения `std::condition_variable` вызовом `notify_one()` или `notify_all()` и вычисления предиката в `true`.

Объявление

```
template<typename Predicate>
void wait(std::unique_lock<std::mutex>& lock, Predicate pred);
```

Предусловия

Выражение `pred()` должно быть допустимым и должно возвращать значение, допускающее преобразование в тип `bool`. Выражение `lock.owns_lock()` должно вычисляться в `true`, а владельцем блокировки `lock` должен быть поток, вызывающий функцию `wait()`.

Результат

Как при вычислении:

```
while(!pred())
{
    wait(lock);
}
```

Выдает

Любое исключение, выданное при вызове `pred`, или исключение `std::system_error`, если результат не может быть достигнут.

ПРИМЕЧАНИЕ

Возможность ложных пробуждений означает, что количество вызовов `pred` не определено. Мьютекс, ссылающийся на блокировку, будет заблокирован при каждом вызове `pred` и выход из функции произойдет только при условии, что в результате вычисления `(bool)pred()` будет возвращено значение `true`.

Синхронизация

Вызовы `notify_one()`, `notify_all()`, `wait()`, `wait_for()` и `wait_until()` в отношении одного и того же экземпляра `std::condition_variable` выстраиваются в последовательность. Вызов `notify_one()` или `notify_all()` разбудит только те потоки, которые приступили к ожиданию до этого вызова.

Компонентная функция `std::condition_variable::wait_for`

Ожидает, пока `std::condition_variable` не будет оповещена вызовом `notify_one()` или `notify_all()`, или пока не истечет определенный период времени, или пока поток не получит ложное пробуждение.

Объявление

```
template<typename Rep, typename Period>
cv_status wait_for(
```

```
std::unique_lock<std::mutex>& lock,
std::chrono::duration<Rep,Period> const& relative_time);
```

Предусловия

`lock.owns_lock()` возвращает `true`, и блокировка принадлежит вызывающему потоку.

Результат

Атомарно снимает блокировку с предоставленного объекта `lock` и блокирует поток, пока он не будет разбужен вызовом `notify_one()` или `notify_all()` из другого потока, или же пока не пройдет период времени, заданный аргументом `relative_time`, или же пока не произойдет ложное пробуждение потока. Перед возвратом управления из вызова `wait_for()` объект `lock` снова блокируется.

Возвращает

`std::cv_status::no_timeout`, если поток был разбужен вызовом `notify_one()`, вызовом `notify_all()` или в результате ложного пробуждения, в противном случае возвращает `std::cv_status::timeout`.

Выдает

Исключение `std::system_error`, если результат не может быть достигнут. Если объект `lock` был разблокирован в ходе вызова `wait_for()`, то при выходе он снова блокируется, даже если выход из функции происходит по выдаче исключения.

ПРИМЕЧАНИЕ

Ложные пробуждения означают, что вызывавший функцию `wait_for()` поток можно разбудить, даже если ни один из потоков не вызывал функцию `notify_one()` или `notify_all()`. Поэтому по возможности рекомендуется применять переопределение функции `wait_for()`, принимающее предикат. Или же рекомендуется вызывать `wait_for()` в цикле, проверяющем предикат, связанный с условной переменной. При работе с этой функцией особое внимание нужно обратить на допустимость тайм-аута; при многих обстоятельствах более подходящей может оказаться функция `wait_until()`. Поток можно заблокировать на время дольше указанного. По возможности истекшее время нужно измерять по стабильным часам.

Синхронизация

Вызовы `notify_one()`, `notify_all()`, `wait()`, `wait_for()` и `wait_until()` в отношении одного и того же экземпляра `std::condition_variable` выстраиваются в последовательность. Вызов `notify_one()` или `notify_all()` разбудит только те потоки, которые приступили к ожиданию до этого вызова.

Переопределение компонентной функции `std::condition_variable::wait_for`, принимающее предикат

Ожидает, пока `std::condition_variable` не будет оповещена вызовом `notify_one()` или `notify_all()` и предикат не будет вычислен в `true` или пока не истечет определенный период времени.

Объявление

```
template<typename Rep,typename Period,typename Predicate>
bool wait_for(
    std::unique_lock<std::mutex>& lock,
    std::chrono::duration<Rep,Period> const& relative_time,
    Predicate pred);
```

Предусловия

Выражение `pred()` должно быть допустимым и должно возвращать значение, допускающее преобразование в тип `bool`. Выражение `lock.owns_lock()` должно вычисляться в `true`, а владельцем блокировки `lock` должен быть поток, вызывающий функцию `wait_for()`.

Результат

Как при вычислении

```
internal_clock::time_point end=internal_clock::now()+relative_time;
while(!pred())
{
    std::chrono::duration<Rep,Period> remaining_time=
        end-internal_clock::now();
    if(wait_for(lock,remaining_time)==std::cv_status::timeout)
        return pred();
}
return true;
```

Возвращает

`true`, если самый последний вызов `pred()` завершился возвращением `true`, или `false`, если период времени, указанный с помощью `relative_time`, истек и функция `pred()` вернула `false`.

ПРИМЕЧАНИЕ

Возможность ложных пробуждений означает, что количество вызовов `pred` не определено. Мьютекс, ссылающийся на блокировку, будет заблокирован при каждом вызове `pred`, и выход из функции произойдет только при условии, что в результате вычисления `(bool)pred()` будет возвращено значение `true` или период времени, указанный с помощью `relative_time`, истек. Поток может быть заблокирован на время дольше указанного. По возможности истекшее время нужно измерять по стабильным часам.

Выдает

Любое исключение, выданное при вызове `pred`, или исключение `std::system_error`, если результат не может быть достигнут.

Синхронизация

Вызовы `notify_one()`, `notify_all()`, `wait()`, `wait_for()` и `wait_until()` в отношении одного и того же экземпляра `std::condition_variable` выстраиваются в последовательность. Вызов `notify_one()` или `notify_all()` разбудит только те потоки, которые приступили к ожиданию до этого вызова.

Компонентная функция `std::condition_variable::wait_until`

Ожидает, пока `std::condition_variable` не будет оповещена вызовом `notify_one()` или `notify_all()`, пока не наступит указанный момент времени или пока поток не получит ложное пробуждение.

Объявление

```
template<typename Clock, typename Duration>
cv_status wait_until(
    std::unique_lock<std::mutex>& lock,
    std::chrono::time_point<Clock, Duration> const& absolute_time);
```

Предусловия

`lock.owns_lock()` возвращает `true`, и блокировка принадлежит вызывающему потоку.

Результат

Атомарно снимает блокировку с предоставленного объекта `lock` и блокирует поток, пока он не будет разбужен вызовом `notify_one()` или `notify_all()` из другого потока, или же пока вызовом `Clock::now()` не будет возвращено время, равное или большее значения `absolute_time`, или же пока не произойдет ложное пробуждение потока. Перед возвратом управления из вызова `wait_until()` объект `lock` снова блокируется.

Возвращает

`std::cv_status::no_timeout`, если поток был разбужен вызовом `notify_one()`, вызовом `notify_all()` или в результате ложного пробуждения, в противном случае возвращает `std::cv_status::timeout`.

Выдает

Исключение `std::system_error`, если результат не может быть достигнут. Если объект `lock` был разблокирован в ходе вызова `wait_until()`, то при выходе он снова блокируется, даже если выход из функции происходит по выдаче исключения.

ПРИМЕЧАНИЕ

Ложные пробуждения означают, что вызывавший функцию `wait_until()` поток можно разбудить, даже если ни один из потоков не вызывал функцию `notify_one()` или `notify_all()`. Поэтому по возможности рекомендуется применять переопределение функции `wait_until()`, принимающее предикат. Или же рекомендуется вызывать `wait_until()` в цикле, проверяющем предикат, связанный с условной переменной. Насчет продолжительности блокировки не дается никаких гарантий, только если функция вернула `false`, вызов `Clock::now()` возвращает время, равное или большее `absolute_time` в том месте, где поток стал разблокированным.

Синхронизация

Вызовы `notify_one()`, `notify_all()`, `wait()`, `wait_for()` и `wait_until()` в отношении одного и того же экземпляра `std::condition_variable` выстраиваются в последовательность. Вызов `notify_one()` или `notify_all()` разбудит только те потоки, которые приступили к ожиданию до этого вызова.

Переопределение компонентной функции `std::condition_variable::wait_until`, принимающее предикат

Ожидает, пока `std::condition_variable` не будет оповещена вызовом `notify_one()` или `notify_all()` и предикат не будет вычислен в `true` или пока не наступит указанный момент времени.

Объявление

```
template<typename Clock, typename Duration, typename Predicate>
bool wait_until(
    std::unique_lock<std::mutex>& lock,
    std::chrono::time_point<Clock, Duration> const& absolute_time,
    Predicate pred);
```

Предусловия

Выражение `pred()` должно быть допустимым и должно возвращать значение, допускающее преобразование в тип `bool`. Выражение `lock.owns_lock()` должно вычисляться в `true`, а владельцем блокировки `lock` должен быть поток, вызывающий функцию `wait()`.

Результат

Как при вычислении

```
while(!pred())
{
    if(wait_until(lock, absolute_time) == std::cv_status::timeout)
        return pred();
}
return true;
```

Возвращает

`true`, если самый последний вызов `pred()` завершился возвращением `true`, или `false`, если в результате вызова `Clock::now()` было возвращено время, равное или большее времени, указанного с помощью `absolute_time`, и функция `pred()` вернула `false`.

ПРИМЕЧАНИЕ

Возможность ложных пробуждений означает, что количество вызовов `pred` не определено. Мьютекс, ссылающийся на блокировку, будет заблокирован при каждом вызове `pred`, и выход из функции произойдет только при условии, что в результате вычисления `(bool)pred()` будет возвращено значение `true`, или в результате вызова `Clock::now()` будет возвращено время, равное или большее времени, указанного с помощью `absolute_time`. Насчет продолжительности блокировки не дается никаких гарантий, только если функция вернула `false`, вызов `Clock::now()` возвращает время, равное или большее `absolute_time` в том месте, где поток стал разблокированным.

Выдает

Любое исключение, выданное при вызове `pred`, или исключение `std::system_error`, если результат не может быть достигнут.

Синхронизация

Вызовы `notify_one()`, `notify_all()`, `wait()`, `wait_for()` и `wait_until()` в отношении одного и того же экземпляра `std::condition_variable` выстраиваются в последовательность. Вызов `notify_one()` или `notify_all()` разбудит только те потоки, которые приступили к ожиданию до этого вызова.

Некомпонентная функция `std::notify_all_at_thread_exit`

Пробуждает все потоки, ожидающие определенной условной переменной `std::condition_variable`, при выходе из текущего потока.

Объявление

```
void notify_all_at_thread_exit(
    condition_variable& cv, unique_lock<mutex> lk);
```

Предусловия

`lk.owns_lock()` возвращает `true`, и блокировка принадлежит вызывающему потоку. Функция `lk.mutex()` должна возвращать такое же значение, как для любого объекта блокировки, предоставляемого компонентным функциям `wait()`, `wait_for()` или `wait_until()` объекта `cv` из конкурентно ожидающих потоков.

Результат

Передаёт владение блокировкой, удерживаемой `lk`, во внутреннее хранилище и планирует уведомление `cv` при выходе из текущего потока. Это уведомление должно быть таким же, как и при вычислении

```
lk.unlock();
cv.notify_all();
```

Выдает

Исключение `std::system_error`, если результат не может быть достигнут.

ПРИМЕЧАНИЕ

Блокировка удерживается до выхода из потока, поэтому следует принять меры, позволяющие избежать взаимной блокировки. Рекомендуется как можно более быстрый выход из вызывающего потока и отказ от применения в нем каких-либо блокирующих операций.

Пользователь должен обеспечить невозможность ошибочного предположения со стороны ожидающих потоков факта выхода из вызывающего потока при их пробуждении, в частности по причине ложных пробуждений. Этой цели можно добиться путем тестирования в ожидающем потоке предиката, который приобретает значение `true` только в уведомляющем потоке, находящемся под защитой мьютекса и без снятия блокировки мьютекса до вызова `notify_all_at_thread_exit`.

Г.2.2. Класс `std::condition_variable_any`

Класс `std::condition_variable_any` позволяет потоку ожидать вычисления условия в `true`. Объект `std::condition_variable` может использоваться только с блокиров-

кой типа `std::unique_lock<std::mutex>`, а объект `std::condition_variable_any` может использоваться с любым типом блокировки, отвечающим требованиям концепции `Lockable`.

Экземпляры `std::condition_variable_any` не являются `CopyAssignable`, `CopyConstructible`, `MoveAssignable` и `MoveConstructible`.

Определение класса

```
class condition_variable_any
{
public:
    condition_variable_any();
    ~condition_variable_any();

    condition_variable_any(
        condition_variable_any const& ) = delete;

    condition_variable_any& operator=(
        condition_variable_any const& ) = delete;

    void notify_one() noexcept;
    void notify_all() noexcept;

    template<typename Lockable>
    void wait(Lockable& lock);

    template <typename Lockable, typename Predicate>
    void wait(Lockable& lock, Predicate pred);

    template <typename Lockable, typename Clock, typename Duration>
    std::cv_status wait_until(
        Lockable& lock,
        const std::chrono::time_point<Clock, Duration>& absolute_time);

    template <
        typename Lockable, typename Clock,
        typename Duration, typename Predicate>
    bool wait_until(
        Lockable& lock,
        const std::chrono::time_point<Clock, Duration>& absolute_time,
        Predicate pred);

    template <typename Lockable, typename Rep, typename Period>
    std::cv_status wait_for(
        Lockable& lock,
        const std::chrono::duration<Rep, Period>& relative_time);

    template <
        typename Lockable, typename Rep,
        typename Period, typename Predicate>
    bool wait_for(
        Lockable& lock,
        const std::chrono::duration<Rep, Period>& relative_time,
        Predicate pred);
};
```

Конструктор по умолчанию `std::condition_variable_any`

Создает объект `std::condition_variable_any`.

Объявление

```
condition_variable_any();
```

Результат

Создает новый экземпляр `std::condition_variable_any`.

Выдает

Исключение `std::system_error`, если условная переменная не может быть создана.

Деструктор `std::condition_variable_any`

Уничтожает объект `std::condition_variable_any`.

Объявление

```
~condition_variable_any();
```

Предусловия

Отсутствие потоков, заблокированных на `*this` в вызове `wait()`, `wait_for()` или `wait_until()`.

Результат

Уничтожает `*this`.

Выдает

Ничего не выдает.

Компонентная функция `std::condition_variable_any::notify_one`

Пробуждает один из потоков, ожидающих определенной условной переменной `std::condition_variable_any`.

Объявление

```
void notify_one() noexcept;
```

Результат

Пробуждает в точке вызова один из потоков, ожидающих `*this`. При отсутствии ожидающих потоков вызов не даст никакого результата.

Выдает

Исключение `std::system_error`, если результат не может быть достигнут.

Синхронизация

Вызовы `notify_one()`, `notify_all()`, `wait()`, `wait_for()` и `wait_until()` в отношении одного и того же экземпляра `std::condition_variable_any` выстраиваются

в последовательность. Вызов `notify_one()` или `notify_all()` разбудит только те потоки, которые приступили к ожиданию до этого вызова.

Компонентная функция `std::condition_variable_any::notify_all`

Пробуждает все потоки, ожидающие определенной условной переменной `std::condition_variable_any`.

Объявление

```
void notify_all() noexcept;
```

Результат

Пробуждает в точке вызова все потоки, ожидающие `*this`. При отсутствии ожидающих потоков вызов не дает никакого результата.

Выдает

Исключение `std::system_error`, если результат не может быть достигнут.

Синхронизация

Вызовы `notify_one()`, `notify_all()`, `wait()`, `wait_for()` и `wait_until()` в отношении одного и того же экземпляра `std::condition_variable_any` выстраиваются в последовательность. Вызов `notify_one()` или `notify_all()` разбудит только те потоки, которые приступили к ожиданию до этого вызова.

Компонентная функция `std::condition_variable_any::wait`

Ожидает, пока `std::condition_variable_any` не будет оповещена вызовом `notify_one()` или `notify_all()` или пока поток не получит ложное пробуждение.

Объявление

```
template<typename Lockable>
void wait(Lockable& lock);
```

Предусловия

Тип `Lockable` является `Lockable`, блокировкой владеет объект `lock`.

Результат

Атомарно снимает блокировку с предоставленного объекта `lock` и блокирует поток, пока он не будет разбужен вызовом `notify_one()` или `notify_all()` из другого потока или же пока не произойдет ложное пробуждение потока. Перед возвратом управления из вызова `wait()` объект `lock` снова блокируется.

Выдает

Исключение `std::system_error`, если результат не может быть достигнут. Если объект `lock` был разблокирован в ходе вызова `wait()`, то при выходе он снова блокируется, даже если выход из функции происходит по выдаче исключения.

ПРИМЕЧАНИЕ

Ложные пробуждения означают, что вызывавший функцию `wait()` поток можно разбудить, даже если ни один из потоков не вызывал функцию `notify_one()` или `notify_all()`. Поэтому по возможности рекомендуется применять переопределение функции `wait()`, принимающее предикат. Или же рекомендуется вызывать `wait()` в цикле, проверяющем предикат, связанный с условной переменной.

Синхронизация

Вызовы `notify_one()`, `notify_all()`, `wait()`, `wait_for()` и `wait_until()` в отношении одного и того же экземпляра `std::condition_variable_any` выстраиваются в последовательность. Вызов `notify_one()` или `notify_all()` разбудит только те потоки, которые приступили к ожиданию до этого вызова.

Переопределение компонентной функции**`std::condition_variable_any::wait`, принимающее предикат**

Ожидает пробуждения `std::condition_variable_any` вызовом `notify_one()` или `notify_all()` и вычисления предиката в `true`.

Объявление

```
template<typename Lockable, typename Predicate>
void wait(Lockable& lock, Predicate pred);
```

Предусловия

Выражение `pred()` должно быть допустимым и должно возвращать значение, допускающее преобразование в тип `bool`. Тип `Lockable` является `Lockable`, и блокировкой владеет объект `lock`.

Результат

Как при вычислении

```
while(!pred())
{
    wait(lock);
}
```

Выдает

Любое исключение, выданное при вызове `pred`, или исключение `std::system_error`, если результат не может быть достигнут.

ПРИМЕЧАНИЕ

Возможность ложных пробуждений означает, что количество вызовов `pred` не определено. Мьютекс, ссылающийся на блокировку, будет заблокирован при каждом вызове `pred`, и выход из функции произойдет только при условии, что в результате вычисления `(bool)pred()` будет возвращено значение `true`.

Синхронизация

Вызовы `notify_one()`, `notify_all()`, `wait()`, `wait_for()` и `wait_until()` в отношении одного и того же экземпляра `std::condition_variable` выстраиваются в по-

следовательность. Вызов `notify_one()` или `notify_all()` разбудит только те потоки, которые приступили к ожиданию до этого вызова.

Компонентная функция `std::condition_variable_any::wait_for`

Ожидает, пока `std::condition_variable_any` не будет оповещена вызовом `notify_one()` или `notify_all()`, или пока не истечет определенный период времени, или пока поток не получит ложное пробуждение.

Объявление

```
template<typename Lockable, typename Rep, typename Period>
std::cv_status wait_for(
    Lockable& lock,
    std::chrono::duration<Rep, Period> const& relative_time);
```

Предусловия

Тип `Lockable` является `Lockable`, и блокировкой владеет объект `lock`.

Результат

Атомарно снимает блокировку с предоставленного объекта `lock` и блокирует поток, пока он не будет разбужен вызовом `notify_one()` или `notify_all()` из другого потока, или же пока не пройдет период времени, заданный аргументом `relative_time`, или же пока не произойдет ложное пробуждение потока. Перед возвратом управления из вызова `wait_for()` объект `lock` снова блокируется.

Возвращает

`std::cv_status::no_timeout`, если поток был разбужен вызовом `notify_one()`, вызовом `notify_all()` или в результате ложного пробуждения, в противном случае возвращает `std::cv_status::timeout`.

Выдает

Исключение `std::system_error`, если результат не может быть достигнут. Если объект `lock` был разблокирован в ходе вызова `wait_for()`, то при выходе он снова блокируется, даже если выход из функции происходит по выдаче исключения.

ПРИМЕЧАНИЕ

Ложные пробуждения означают, что вызывавший функцию `wait_for()` поток можно разбудить, даже если ни один из потоков не вызывал функцию `notify_one()` или `notify_all()`. Поэтому по возможности рекомендуется применять переопределение функции `wait_for()`, принимающее предикат. Или же рекомендуется вызывать `wait_for()` в цикле, проверяющем предикат, связанный с условной переменной. При работе с этой функцией особое внимание нужно обратить на допустимость тайм-аута; при многих обстоятельствах более подходящей может оказаться функция `wait_until()`. Поток может быть заблокирован на время дольше указанного. По возможности истекшее время нужно измерять по стабильным часам.

Синхронизация

Вызовы `notify_one()`, `notify_all()`, `wait()`, `wait_for()` и `wait_until()` в отношении одного и того же экземпляра `std::condition_variable` выстраиваются в последовательность. Вызов `notify_one()` или `notify_all()` разбудит только те потоки, которые приступили к ожиданию до этого вызова.

Переопределение компонентной функции**`std::condition_variable_any::wait_for`, принимающее предикат**

Ожидает, пока `std::condition_variable_any` не будет оповещена вызовом `notify_one()` или `notify_all()` и предикат не будет вычислен в `true` или пока не истечет определенный период времени.

Объявление

```
template<typename Lockable, typename Rep,
        typename Period, typename Predicate>
bool wait_for(
    Lockable& lock,
    std::chrono::duration<Rep, Period> const& relative_time,
    Predicate pred);
```

Предусловия

Выражение `pred()` должно быть допустимым и должно возвращать значение, допускающее преобразование в тип `bool`. Тип `Lockable` является `Lockable`, и блокировкой владеет объект `lock`.

Результат

Как при вычислении

```
internal_clock::time_point end=internal_clock::now()+relative_time;
while(!pred())
{
    std::chrono::duration<Rep, Period> remaining_time=
        end-internal_clock::now();
    if(wait_for(lock, remaining_time)==std::cv_status::timeout)
        return pred();
}
return true;
```

Возвращает

`true`, если самый последний вызов `pred()` завершился возвращением `true`, или `false`, если период времени, указанный с помощью `relative_time`, истек и функция `pred()` вернула `false`.

ПРИМЕЧАНИЕ

Возможность ложных пробуждений означает, что количество вызовов `pred` не определено. Мьютекс, ссылающийся на блокировку, будет заблокирован при каждом вызове `pred`, и выход из функции произойдет только при условии,

что в результате вычисления (bool)pred() будет возвращено значение true, или период времени, указанный с помощью relative_time, истек. Поток может быть заблокирован на время дольше указанного. По возможности истекшее время нужно измерять по стабильным часам.

Выдает

Любое исключение, выданное при вызове pred, или исключение std::system_error, если результат не может быть достигнут.

Синхронизация

Вызовы notify_one(), notify_all(), wait(), wait_for() и wait_until() в отношении одного и того же экземпляра std::condition_variable выстраиваются в последовательность. Вызов notify_one() или notify_all() разбудит только те потоки, которые приступили к ожиданию до этого вызова.

Компонентная функция std::condition_variable_any::wait_until

Ожидает, пока std::condition_variable_any не будет оповещена вызовом notify_one() или notify_all(), пока не наступит указанный момент времени или пока поток не получит ложное пробуждение.

Объявление

```
template<typename Lockable, typename Clock, typename Duration>
std::cv_status wait_until(
    Lockable& lock,
    std::chrono::time_point<Clock, Duration> const& absolute_time);
```

Предусловия

Тип Lockable является Lockable, и блокировкой владеет объект lock.

Результат

Атомарно снимает блокировку с предоставленного объекта lock и блокирует поток, пока он не будет разбужен вызовом notify_one() или notify_all() из другого потока, или же пока вызовом Clock::now() не будет возвращено время, равное или большее значения absolute_time, или же пока не произойдет ложное пробуждение потока. Перед возвратом управления из вызова wait_until() объект lock снова блокируется.

Возвращает

std::cv_status::no_timeout, если поток был разбужен вызовом notify_one(), вызовом notify_all() или в результате ложного пробуждения, в противном случае возвращает std::cv_status::timeout.

Выдает

Исключение std::system_error, если результат не может быть достигнут. Если объект lock был разблокирован в ходе вызова wait_until(), то при выходе он снова блокируется, даже если выход из функции происходит по выдаче исключения.

ПРИМЕЧАНИЕ

Ложные пробуждения означают, что вызывавший функцию `wait_until()` поток можно разбудить, даже если ни один из потоков не вызывал функцию `notify_one()` или `notify_all()`. Поэтому по возможности рекомендуется применять переопределение функции `wait_until()`, принимающее предикат. Или же рекомендуется вызывать `wait_until()` в цикле, проверяющем предикат, связанный с условной переменной. Насчет продолжительности блокировки не дается никаких гарантий, только если функция вернула `false`, вызов `Clock::now()` возвращает время, равное или большее `absolute_time`, в том месте, где поток стал разблокированным.

Синхронизация

Вызовы `notify_one()`, `notify_all()`, `wait()`, `wait_for()` и `wait_until()` в отношении одного и того же экземпляра `std::condition_variable` выстраиваются в последовательность. Вызов `notify_one()` или `notify_all()` разбудит только те потоки, которые приступили к ожиданию до этого вызова.

Переопределение компонентной функции `std::condition_variable_any::wait_until`, принимающее предикат

Ожидает, пока `std::condition_variable_any` не будет оповещена вызовом `notify_one()`, или `notify_all()` и предикат не будет вычислен в `true`, или пока не наступит указанный момент времени.

Объявление

```
template<typename Lockable, typename Clock,
        typename Duration, typename Predicate>
bool wait_until(
    Lockable& lock,
    std::chrono::time_point<Clock, Duration> const& absolute_time,
    Predicate pred);
```

Предусловия

Выражение `pred()` должно быть допустимым и должно возвращать значение, допускающее преобразование в тип `bool`. Тип `Lockable` является `Lockable`, и блокировкой владеет объект `lock`.

Результат

Как при вычислении

```
while(!pred())
{
    if(wait_until(lock, absolute_time) == std::cv_status::timeout)
        return pred();
}
return true;
```

Возвращает

`true`, если самый последний вызов `pred()` завершился возвращением `true`, или `false`, если в результате вызова `Clock::now()` было возвращено время, равное или большее времени, указанного с помощью `absolute_time`, и функция `pred()` вернула `false`.